



FALL 2026

SEG3102 Software Design and Architecture

Domain-Driven Design: From Strategic to Tactic Design

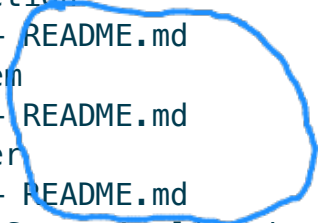
Slide content used in this course is adapted from Prof. Stéphane Somé.

Last lecture: Subdomains

	Competitive advantage	Complexity	Volatility	Implementation	Problem
Core	Yes	High	High	In-house	Interesting
Generic	No	High	Low	Buy or adopt	Solved
Supporting	No	Low	Low	In-house or outsource	Obvious

Last lecture: Subdomain maps to project structure

```
1 auction-system
2   └─ src/main/kotlin/seg3x02.auctionsystem
3       └─ domain
4           ├── auction
5               └─ README.md
6           ├── item
7               └─ README.md
8           └─ user
9               └─ README.md
10  └─ AuctionSystemApplication.kt
```



domain/auction/README.md

```
1 ## Auction Sub-Domain
2
3 - Core
4 - Auctions, Bids
```

Last lecture: Ubiquitous language

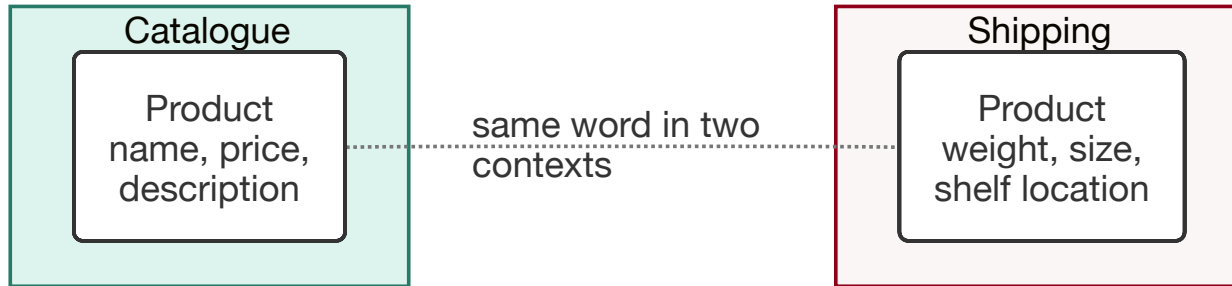
Ubiquitous language

The **ubiquitous language** is the common vocabulary of the domain, shared by domain experts and developers, and used in conversations, documents, models, and code.

- **Common terminology** and structure for the whole team
- Binds the **model and the implementation**
- Supports **change**: when the business changes a term, the code follows

The **domain model** **is** the ubiquitous language. Diagrams, use cases, and classes in code are different views of the same vocabulary.

This lecture: Find boundaries for the ubiquitous language



One ubiquitous language for the whole shop turns out to be impossible.

“Product” means one thing to marketing and another to the warehouse.

These boundaries can be identified with **bounded context**.

Problem domain

↓ decompose

Subdomains

↳ ① group subdomains

② realize them

Outline

1. Bounded contexts
2. Bounded contexts of the online auction system
3. Modeling behavior of a bounded context
4. Modeling behavior of the online auction system



Bounded contexts

Ubiquitous language is hard to create



To the **store** team, an order is what the customer sees: the products bought, the total, an expected delivery date. It is hardly modified after checkout.

To the **warehouse** team, an order is a pick list for the warehouse, which becomes a parcel. Every scan changes its state.



To **accounting**, an order is an invoice with tax and a payment record, paid or refunded. How many parcels it was shipped in is irrelevant.

Option A: one big model

Option B: prefixes

storeOrder, WarehouseOrder,
AccountingOrder, side by side in one
codebase.



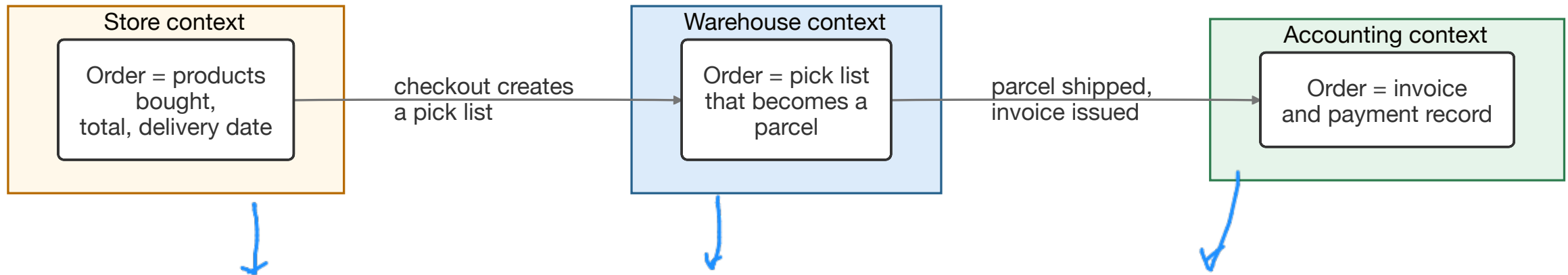
In any case, the model becomes too complex. We need third option.

Recall: The meaning of a language comes from its **context**.

Bounded context

Bounded context

Divide the ubiquitous language into several smaller languages, and assign each one to the explicit **context** in which it applies. That context is the **bounded context**.



A model needs a boundary

A model has a purpose. Without a boundary it grows until it is a copy of the real world, and then it is useless. *(How many if statements do we need to handle all the possible meanings of “order”?)*

A subway map is useless for sailing. It is consistent and useful only **within the scope of its purpose**.

Bounded contexts are the **consistency boundaries** of ubiquitous languages. A term or a rule is only guaranteed to hold inside its context.

Defining the boundary is part of strategic design.



“Ubiquitous” does not mean “universal”

A ubiquitous language is ubiquitous only inside its bounded context.



How big should a bounded context be?

The widest boundary

Consistency of the language sets the **maximum** size: any bigger and two meanings of “order” collide.

Inside a consistent language we may still split further. Where we cut is a **strategic design decision.**

It depends

- Wider: harder to keep consistent
- Smaller: more integration overhead
- Split further for a new team, a separate lifecycle, or to scale one part alone

Never split a **coherent set of use cases**: Return an item and Approve a return change together.

Bounded contexts versus subdomains

Subdomains

- Come from the **business strategy**
- Are **discovered** by analysis
- A set of coherent use cases, defined by the requirements
- The business domain decide them

Problem

Bounded contexts

- Come from the **solution design**
- Are **designed** by us
- The boundary of a model and its language
- A bounded contexts implement multiple subdomains
- Becomes implementation structure.

Solution

Subdomains are discovered. Bounded contexts are designed.

Bounded contexts group subdomains

A bounded context can implement several subdomains.

Store context: what the customer sees

Catalogue subdomain

Product

Category

Ordering subdomain

Cart

Order

Warehouse context: parcels and shelves

Shipping subdomain

Order

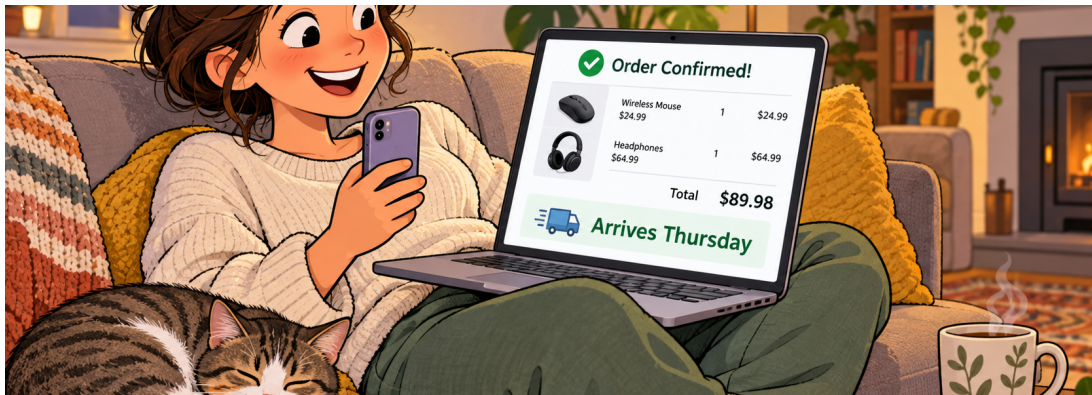
Parcel

Returns subdomain

Return

Inspection

The store team owns browsing and checkout. An **order** is the customer's purchase.

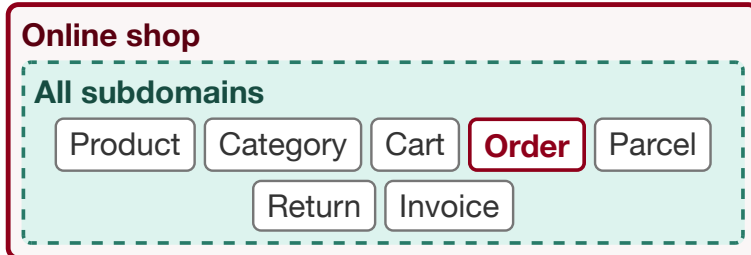


The warehouse team ships parcels and receives them back. An **order** is a pick list.



Different bounded contexts design

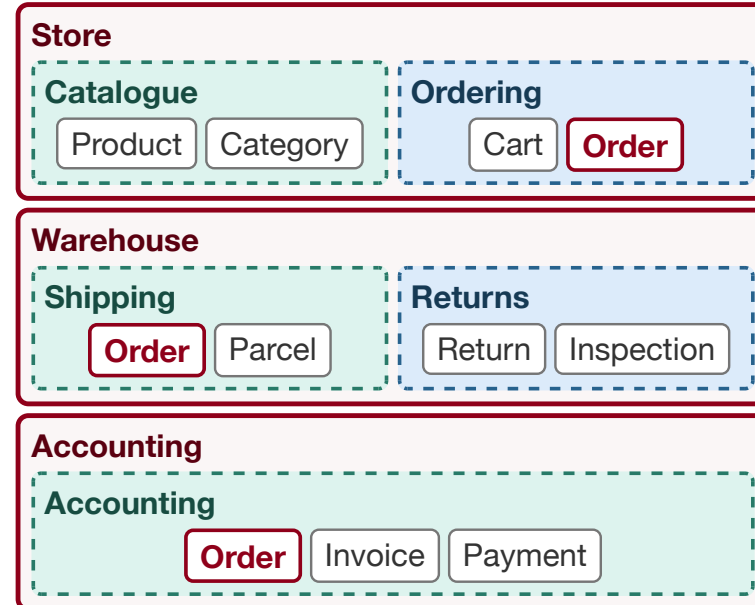
One model



One **Order** for everyone. Efforts need to be coordinated across the whole model.

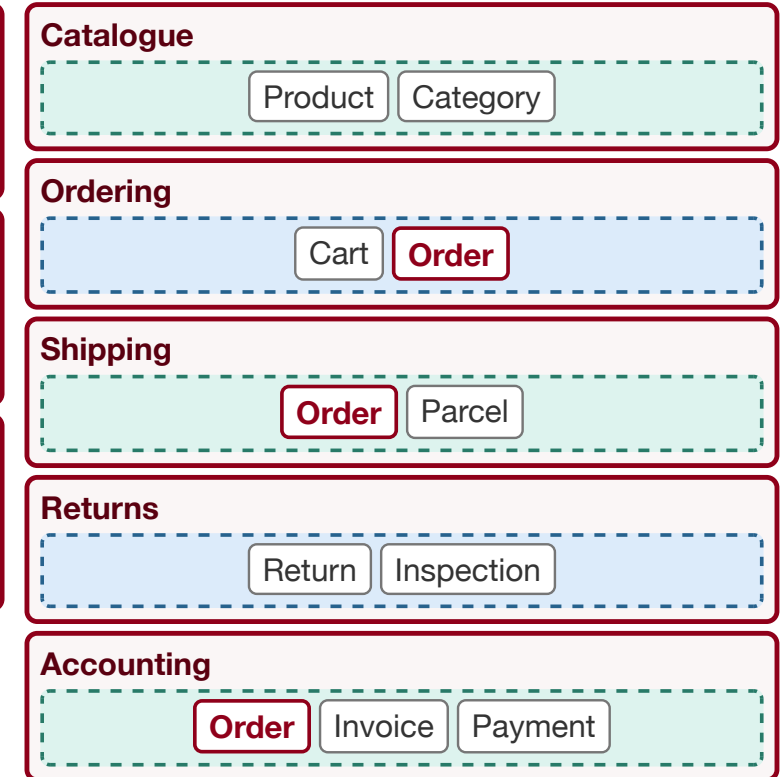
Small

Follow the experts



Split where the experts' models of **Order** conflict: three contexts, three orders.

Per subdomain



Still too big? One context per subdomain.

There is no "correct answer", but only "reasonable answers".

Bounded contexts create boundaries

“Architectural design is system design. System design is contextual design: it is inherently about boundaries (what’s in, what’s out, what spans, what moves between), and about trade-offs. It reshapes what is outside, just as it shapes what is inside.”

Ruth Malan

The bounded context is DDD’s tool for two more kinds of boundary: **physical** and **ownership**.

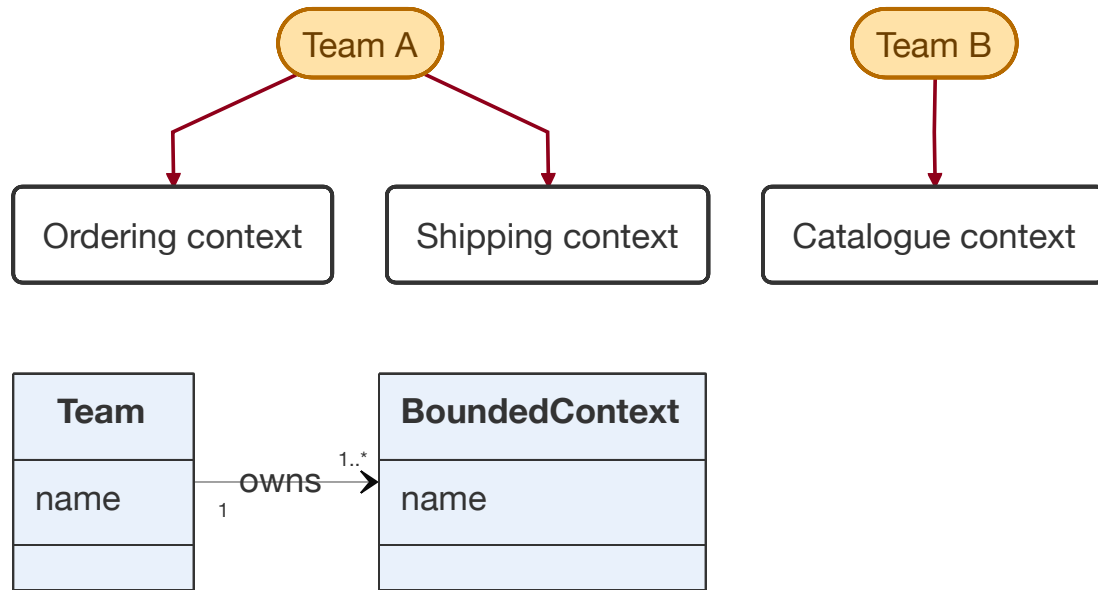
Physical boundaries

- A bounded context is not only a model boundary. Each one can be implemented as its **own service or project** independently.
- Clear physical boundaries let each context use the **design and technology stack** that suits it: concurrency, module, deployment, and so on.

Each context may be a standalone service. Then the context is the **physical** boundary and each subdomain is a **logical** one: a module or package.

1	online-shop/	one deployed service
2	domain/	
3	ordering/	core subdomain
4	catalogue/	supporting
5	account/	generic or supporting
6	application/usecases/	

Ownership boundaries



Say the shop has two development teams. Ownership has to be unambiguous.

- A bounded context is owned by **one team only**
- One team may own several contexts
- Never two teams on one context
-> a source of chaos

Mapping the contexts

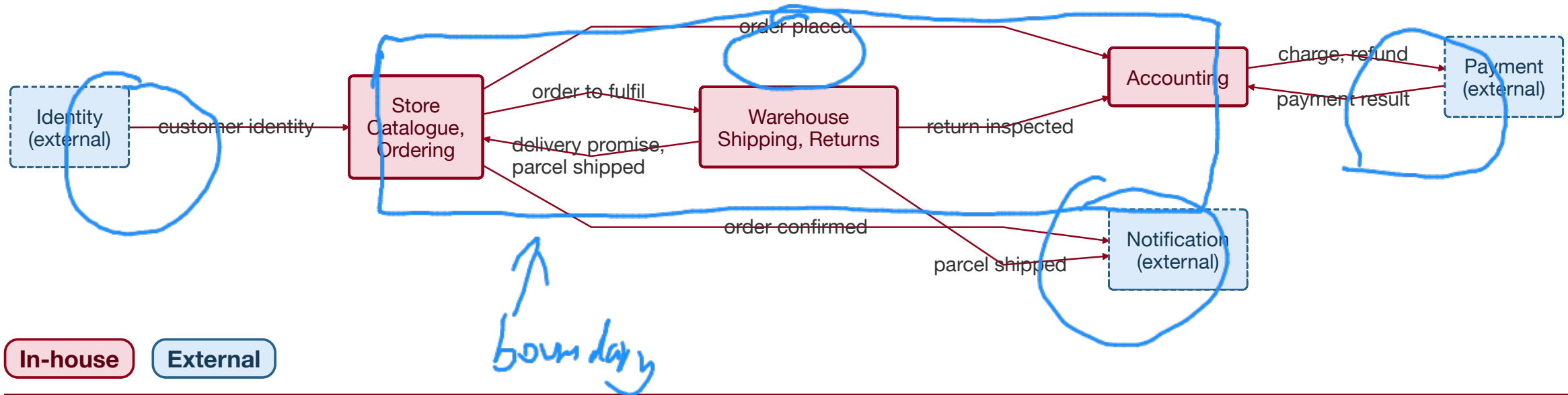
Now we have different teams working on different contexts. They need to **integrate** their work. Integration requires us to know their relationships.

Context diagram

A **context diagram** shows the bounded contexts of a system and the collaborating systems around them, with the relationships between them: which context depends on which, and what crosses the boundary.

- Every bounded context: built **in-house**, **outsourced** to a contractor, or an **external** service integrated into the system
- Crossing a boundary is a design decision, so make it explicit
- Shows each team who it must talk to
- Each line is an integration to design

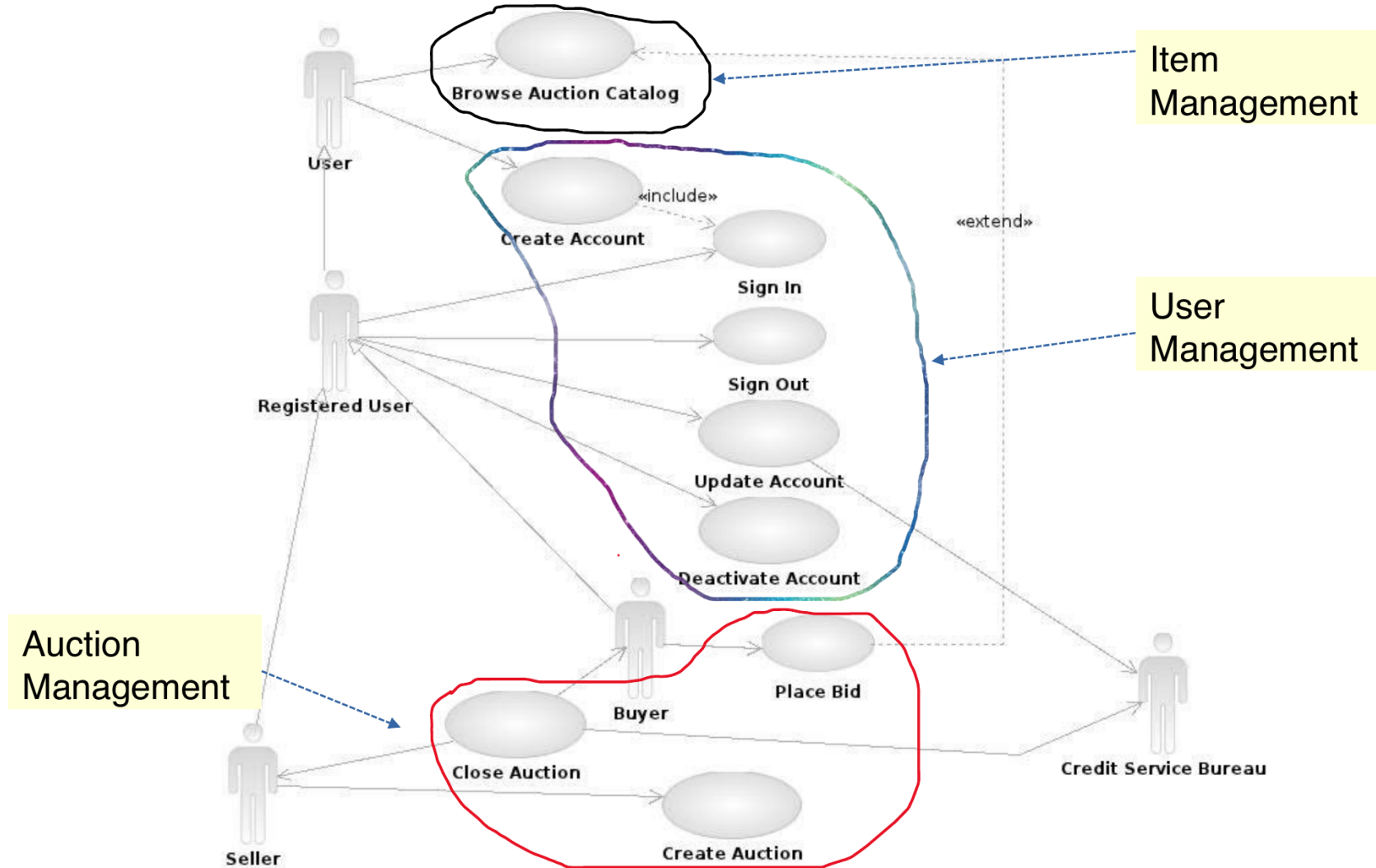
Context diagram of the online shop



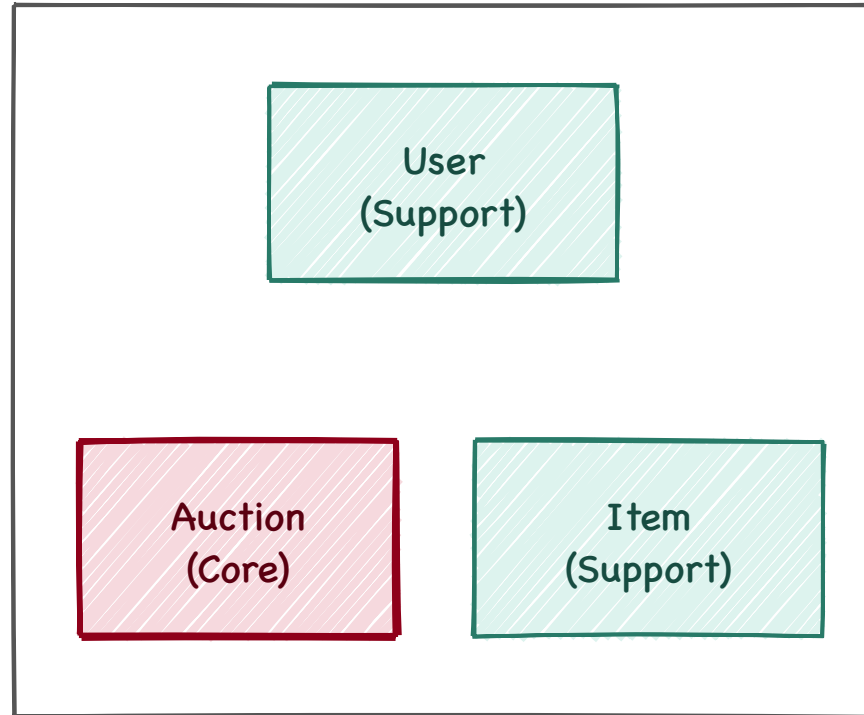
Each arrow hides a question: who adapts to whom, and what happens when the other side changes? Those are the integration patterns. We will see them in later lectures.

Bounded contexts for online auction system

Sub-domains



Sub-domains



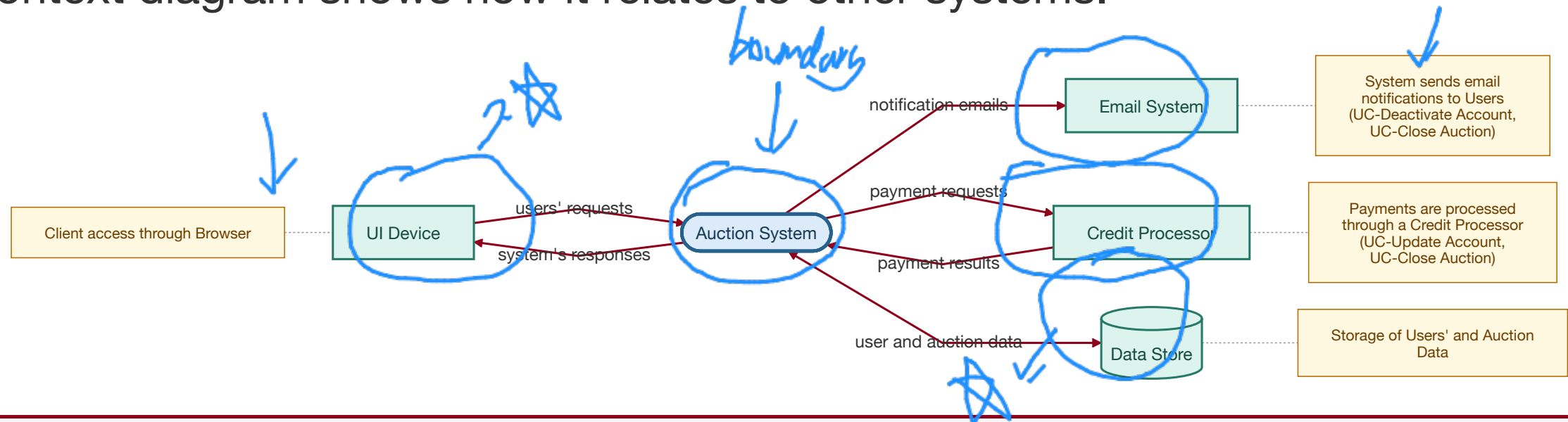
Are there any other subdomains we need but not there?

Context diagram

- System to be built as black box
- Identify collaborating systems in environment
 - multiplicity
 - provide rationale
- Serves to
 - specify system's scope
 - identify external interfaces

The auction system as one bounded context

The auction system fits one model and one language: User, Auction, Bid mean the same thing everywhere. But it does not handle all requirements by itself. A context diagram shows how it relates to other systems.



Note: the data store and the UI are not bounded contexts. They have no model of their own; the store only persists the auction system's model.

Domain modeling: behavior

What happens inside a bounded context?

Inside one context, we want to capture two things:

Behavior

What the system does when an actor asks for something.

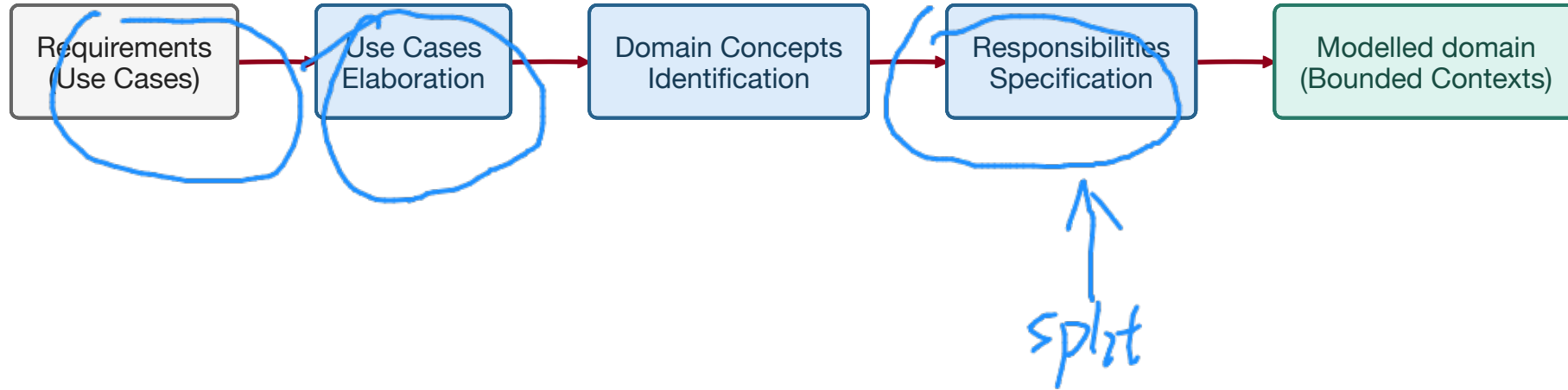
- Use cases and their steps
- Application commands
- Contracts: preconditions and postconditions

Structure

What the system remembers between requests.

- Domain concepts and their attributes
- Associations between them
- Entities, value objects (next lecture)

Domain modelling



Use case elaboration

- To get enough details about use cases
 - pre-conditions / post-conditions
 - interaction flows: scenarios → steps
- Identification of application commands
- User experience design → U-X
- Acceptance testing → ★

A use case interacts with the system



I want to order a wireless mouse and a keyboard. Here is the address.

Before I can do that: is the user signed in? Is the cart empty? Do we have two in stock?

pre



And once it is done: a new order exists, it belongs to the user, the stock is decreased, the cart is empty.

post



A use case reads the system before it acts and may changes it afterwards. Two questions for every use case:

- what must be true before (Precondition)?
- what is different after (Postcondition)?

Start from the use case

Use case: Place an order

Preconditions: Customer is signed in, cart is not empty

Steps

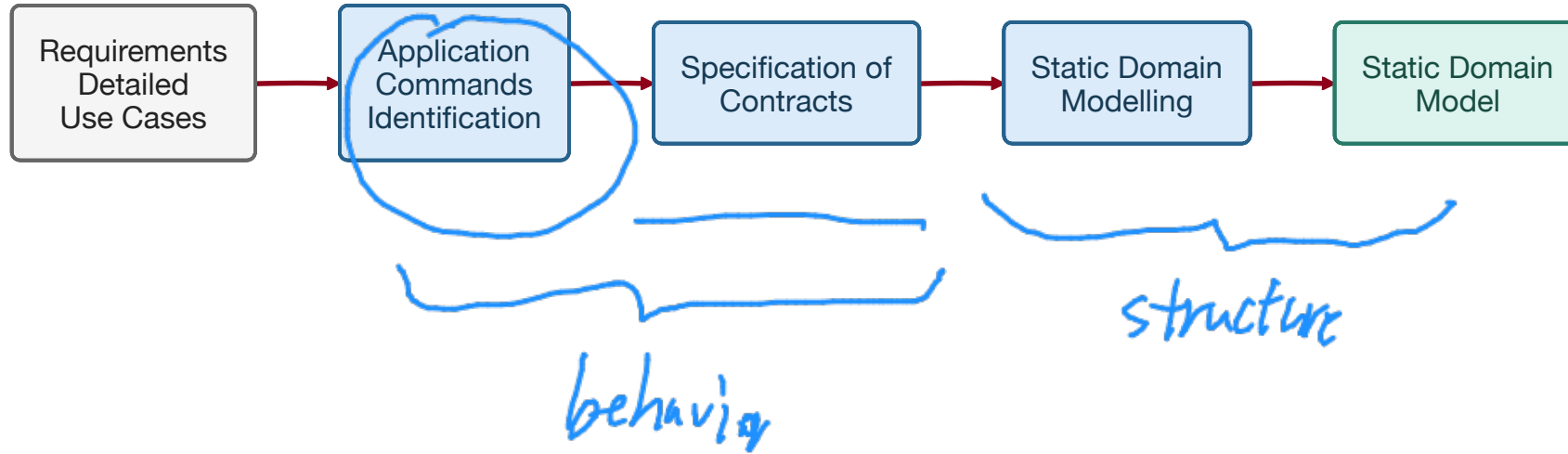
1. Customer selects to check out
2. System shows the cart and asks for a delivery address
3. Customer submits the order information
4. System creates the order, reserves the order items, and adds the order to the Customer's orders

Postcondition: new Order added to the Customer's orders, items reserved in stock

Extension: 4a. an item is not available in the requested quantity, the order is not placed

A use case is written for the reader. The steps mix what the actor does, what the screen shows, and what the model changes. We need to pull those apart.

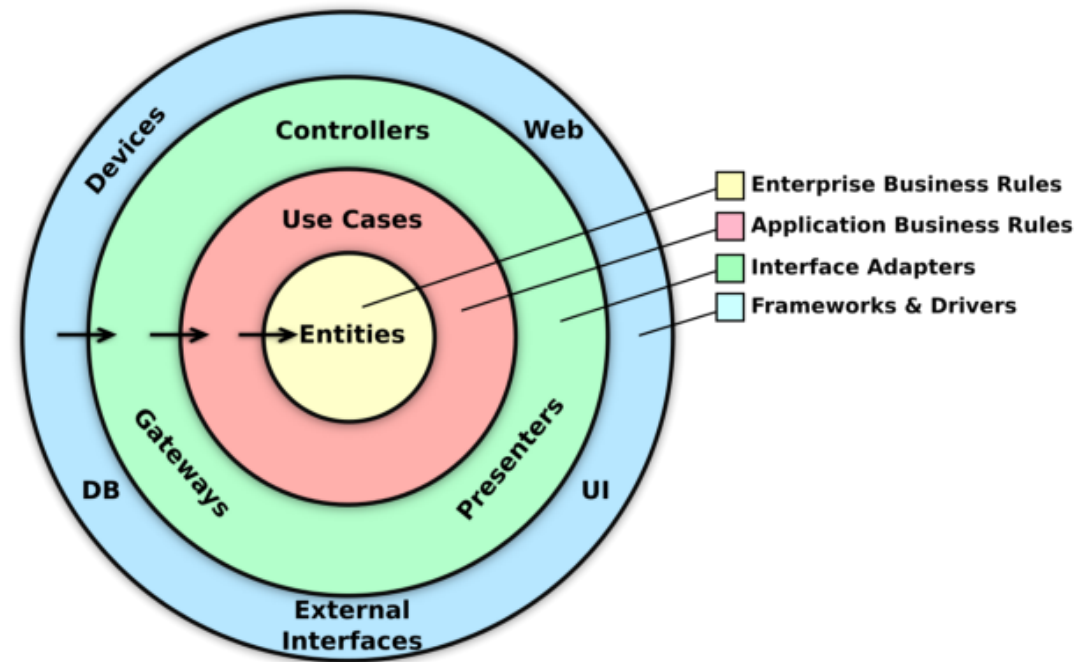
Identification of domain concepts



Application commands

- Command in application layer
 - triggers domain changes
- Identified from use case
 - points where request to system is performed

Application Command



Use Case component handles all commands from a use case

Use case interacts with the model using commands



Find the command

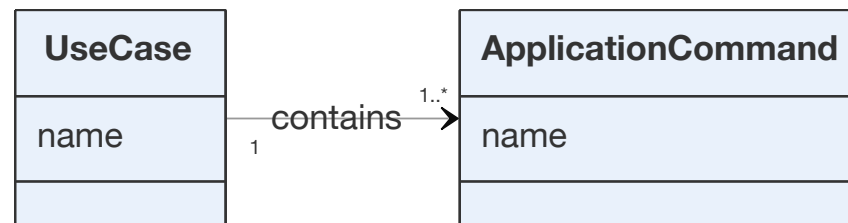
1. Customer selects to check out
2. System shows the cart and asks for a delivery address
3. **Customer submits the order information**
4. System creates the order, reserves the items, and adds the order to the Customer's orders

→ read-only

Application command

An **application command** is the point in a use case where the actor hands the system a request. A command is also a unit you want to test.

Step 2 may also be a (read-only) command.



Name it in the ubiquitous language

Avoid

- `submit()`
- `submitForm()`
- `clickCheckout()`

Name not focused on the intent.

Prefer

- `placeOrder()`

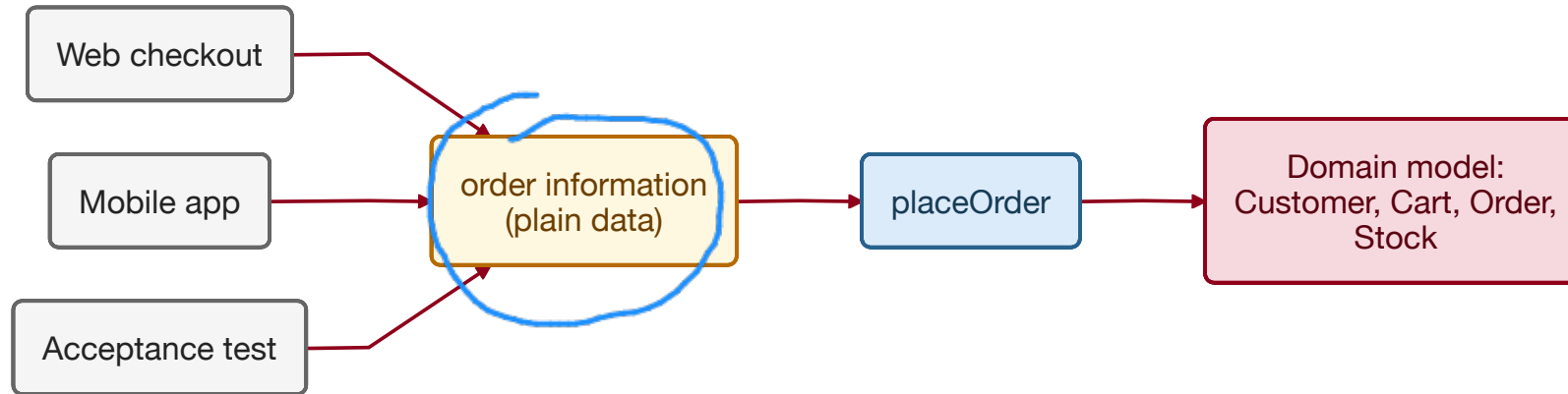
A verb phrase the domain expert already uses.

Test:

- *Would the store team say “the customer places an order”?* Yes.
- *Would they say “the customer submits the form”?*
 - No, this is implementation detail. What if the user orders using a voice assistant?

What crosses the boundary

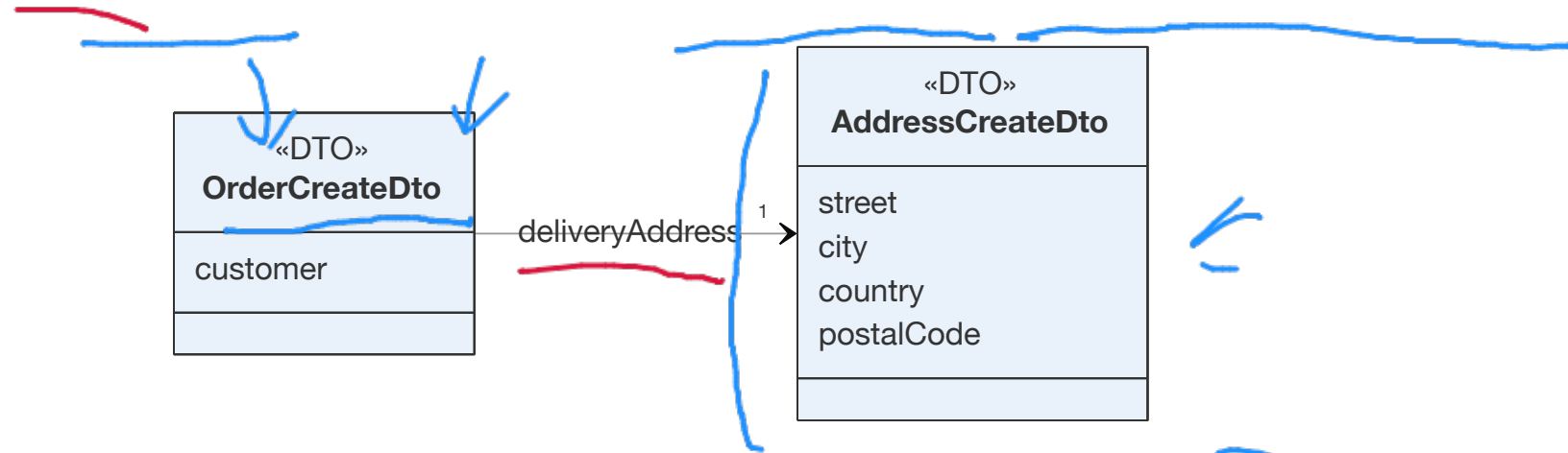
Application commands are independent of the user interface.



The command takes plain data, a **data transfer object**.

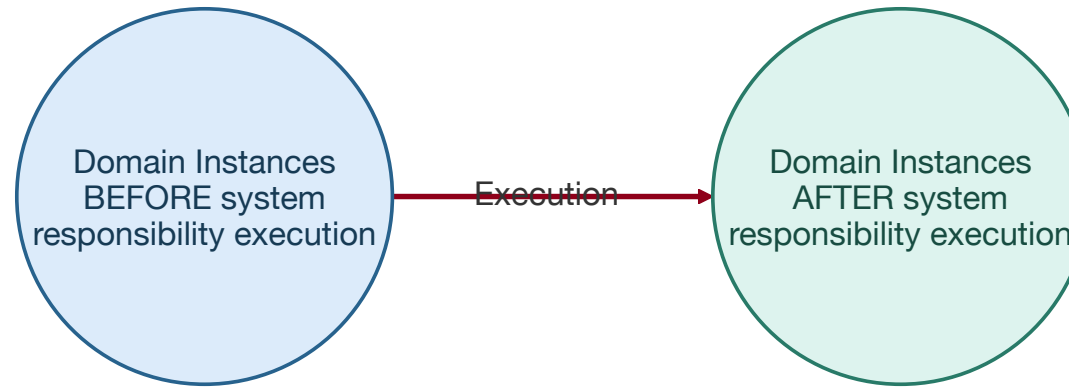
Modeling the data transfer object

Order information: the customer placing the order and the delivery address (street, city, country, postal code).



placeOrder(orderInfo: OrderCreateDto): UUID?

Contract



Preconditions: configuration required for execution

PreCondition

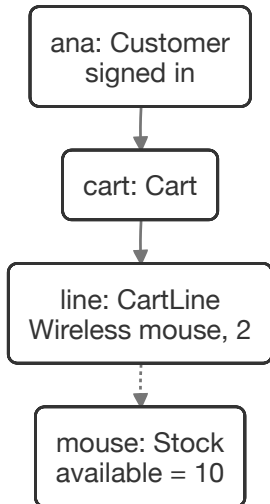
Postconditions: Differences between the configuration before and after

Post Condition

Before and after

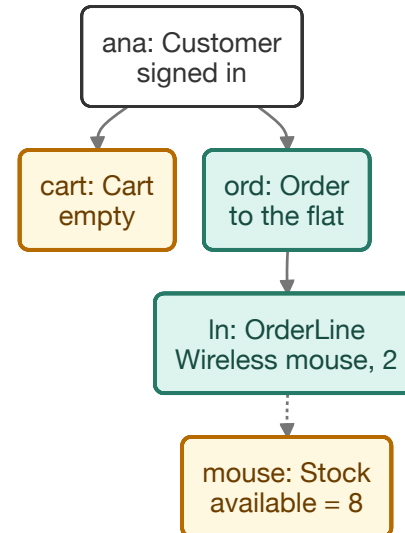
→ Contract

Before `placeOrder`



place
order

After

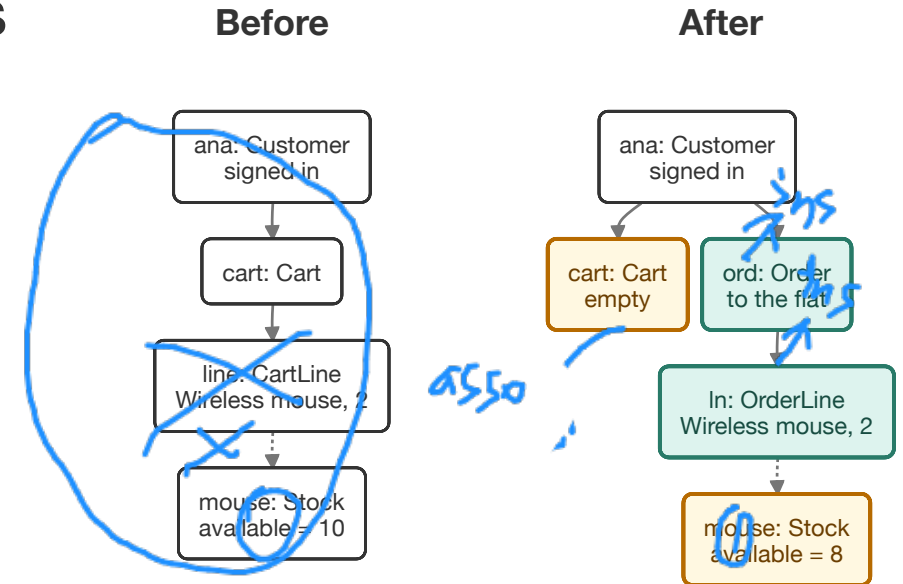


The **contract** defines the precondition and postcondition of a command in terms of the *domain model*

Five kinds of change

Compare the two snapshots. Every difference is one of five kinds:

Change	In the example
Creation of an instance	<i>ord</i> , <i>ln</i>
Destruction of an instance	the cart line
Creation of an association	<i>ana</i> to <i>ord</i> , <i>ord</i> to <i>ln</i>
Destruction of an association	<i>cart</i> to its line
Modification of attributes	stock 10 to 8



Write the contract

Command: `placeOrder(customer, orderInfo)`

Preconditions:

- signed-in `customer` (non null) ✓
- the `customer's` cart has at least one cart line ✓
- for each cart line, the `available stock` covers the quantity ✓

Post-conditions:

- a new Order `ord` was created (creation of instance)
- `ord` was initialized from `orderInfo` (modification of attributes)
- `ord` was added to the `customer's` orders (creation of association)
- for each cart line, a new Order Line `ln` was created from it (creation of instance, modification of attributes)
- `ln` was added to `ord` (creation of association)
- the `available stock` of each product was reduced by the ordered quantity (modification of attributes)
- the `cart lines` were removed from the cart (destruction of instances)

→ `orderInfo`

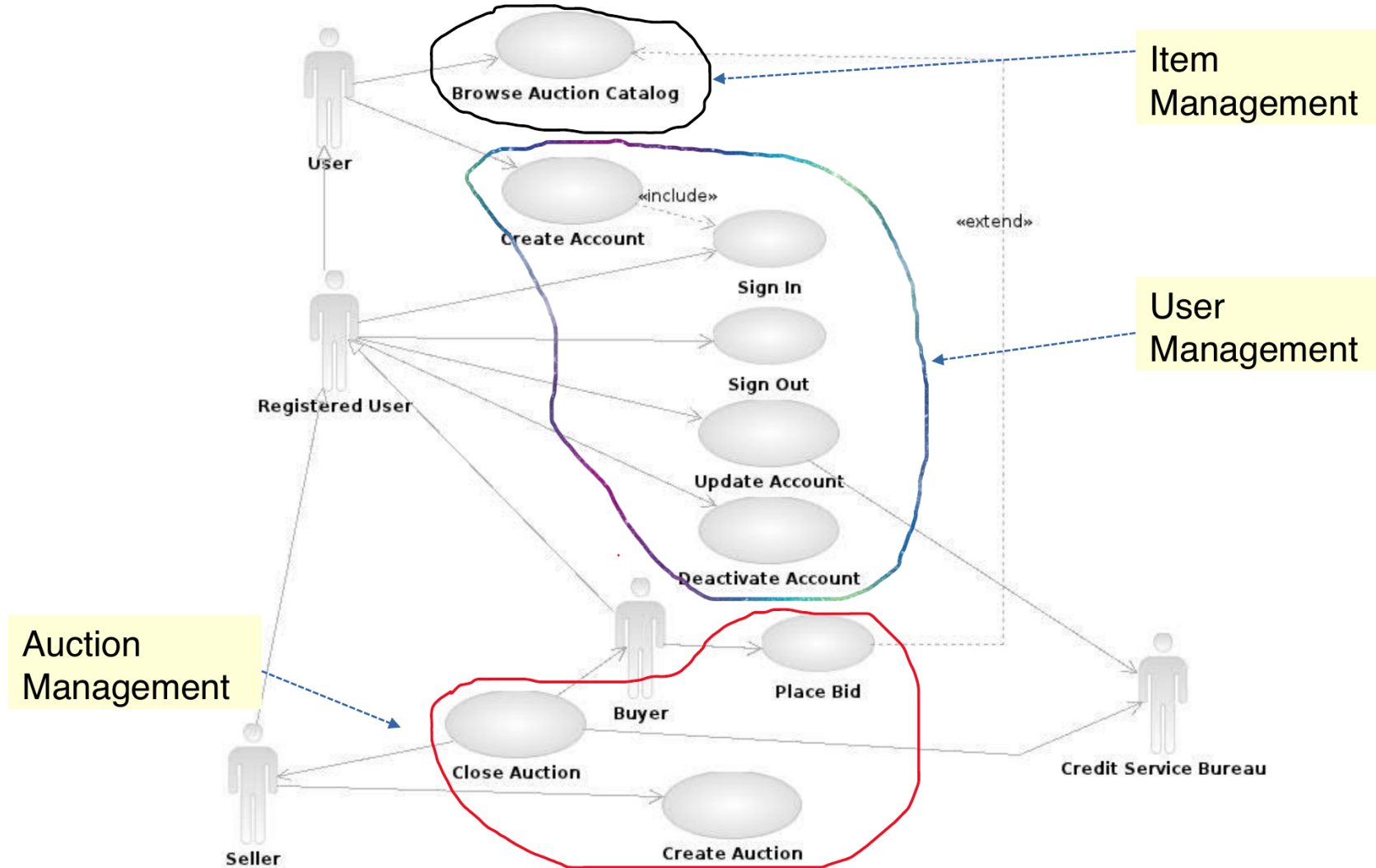
opt! assumption: address is included in the `orderInfo`

opt:
 - add is initialized from
`add DTO` → `COI`
 - add is added to
`orderInfo` → `COA`
`delivery address`

instance · attribute · association

Modeling behavior of the online auction system

Sub-domains



Create Auction

*“The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). Once the auction has been created, Buyers can then place a bid on the item (see the Place Bid use case). To create an auction, **the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case)**”*

Preconditions

Create Auction

“The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). Once the auction has been created, Buyers can then place a bid on the item (see the Place Bid use case). To create an auction, the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case)”

Steps

Create Auction

*“The Create Auction use case allows a Seller to create an online auction by specifying auction information (including the start and end times) and, if it was not provided when the Seller created their user account, credit card information (see the Create Account use case). **Once the auction has been created, Buyers can then place a bid on the item** (see the Place Bid use case). To create an auction, the Seller must be signed in (see the Sign In use case) and must not have any pending payments (see the Close Auction use case)”*

Postcondition

Create Auction

Preconditions:

- Seller is signed in
- Seller doesn't have any pending payment

Steps

1. Seller selects to create an auction
2. System asks for auction information and optional credit card information
3. Seller submits auction information
4. System adds new auction to Seller's list of active auctions

Postcondition:

- New Auction added to Seller's account

Application command

2. System asks for auction information and optional credit card information
3. **Seller submits auction information**
4. System adds new auction to Seller's list of active auctions

Application command

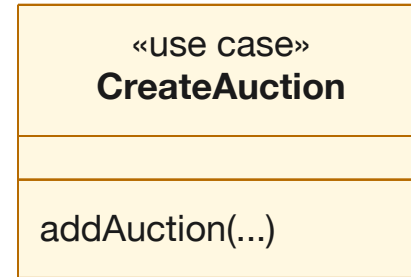
- Name should
 - be abstract, expressed at level of intent
 - independent from input medium, interface widget
 - ideally start with a verb (i.e. add, enter, end, make, ...)

Application command

2. System asks for auction information and optional credit card information
3. **Seller submits auction information**
4. System adds new auction to Seller's list of active auctions
 - What is the intent?
 - What is expected from the System?

Application command

- Good name: `addAuction`
 - reveals intention
 - starts with a verb
 - not linked to a device, input mode



Action Command

2. System asks for auction information and optional credit card information
3. Seller submits auction information
4. System adds new auction to Seller's list of active auctions

Auction Information: captures information about an auction, which includes the start time and duration of the auction, product information (title, description, image), starting price (minimum initial bid price), minimum bid increment, and auction category (the category in which the auction is listed)

Credit Card Information: Information about a credit card, which includes the credit card number, billing Information address, and the card's expiration date.

Data Transfer Object (DTO)

- Carries data between boundaries
 - simple data object
 - information hiding
 - avoids domain pollution
 - better modifiability & evolution

```
1 data class AuctionCreateDto(  
2     val startTime: LocalDateTime,  
3     val duration: Duration,  
4     val startPrice: BigDecimal,  
5     val minIncrement: BigDecimal,  
6     val seller: String,  
7     val category: String,  
8     var itemInfo: ItemCreateDto) {  
9     var creditCardInfo: CreditCardCreateDto? = null  
10 }
```

The application command in the code

application/usecases/CreateAuction.kt

```
1 package seg3x02.auctionsystem.application.usecases
2
3 import java.util.*
4
5 interface CreateAuction {
6     fun addAuction(auctionInfo: AuctionCreateDto): UUID?
7 }
```

The DTOs in the tutorial project

application/dtos/queries/AuctionCreateDto.kt

```
1 package seg3x02.auctionsystem.application.dtos.queries
2
3 import ...
4
5 data class AuctionCreateDto(val startTime: LocalDateTime,
6                             val duration: Duration,
7                             val startPrice: BigDecimal,
8                             val minIncrement: BigDecimal,
9                             val seller: String,
10                            val category: String,
11                            val itemInfo: ItemCreateDto) {
12     var creditCardInfo: CreditCardCreateDto? = null
13 }
```

application/dtos/queries/ItemCreateDto.kt

```
1 package seg3x02.auctionsystem.application.dtos.queries
2
3 data class ItemCreateDto(val title: String,
4                          val description: String) {
5     var image: ByteArray? = null
6 }
```

application/dtos/queries/AddressCreateDto.kt

```
1 package seg3x02.auctionsystem.application.dtos.queries
2
3 data class AddressCreateDto(val street: String,
4                             val city: String,
5                             val country: String,
6                             val postalCode: String)
```

application/dtos/queries/CreditCardCreateDto.kt

```
1 package seg3x02.auctionsystem.application.dtos.queries
2
3 import ...
4
5 data class CreditCardCreateDto(
6     val number: String,
7     val expirationMonth: Month,
8     val expirationYear: Year,
9     val accountFirstname: String,
10    val accountLastname: String,
11    val accountAddress: AddressCreateDto)
```

Contracts

- Specification of Command
 - Preconditions
 - Pre-existing state
 - Postconditions
 - State after execution

addAuction

2. System asks for auction information and optional credit card information

3. Seller submits auction information

4. **System adds new auction to Seller's list of active auctions**

- Creation of Auction
 - Setup based on arguments
 - Association to Seller
- Creation of Auction Item
 - Setup based on arguments
 - Association to Auction
- Creation of Credit Card (if information provided)
 - Setup based on arguments
 - Association to Seller
 - Association to Auction

addAuction

Preconditions:

- Seller is signed in
- Seller doesn't have any pending payment

Preexisting State:

- Current Seller
- Current Seller with no Pending Payment

Command: addAuction(seller, auctionInfo)

Preconditions:

- signed-in **seller** (non null)
- non-existing Pending Payment for Seller

Post-conditions:

- a new Auction **auc** was created (creation of instance)
- **auc** was initialized from **auctionInfo** (modification of attributes)
- **auc** was added to **seller auctions** (creation of association)
- new Auction Item **aucIt** was created (creation of instance)
- **aucIt** was initialized from **auctionInfo** (modification of attributes)
- **aucIt** was set as **item** for **auc** (creation of association)
- **If** credit card infos were provided in **auctionInfo**
 - a new Credit Card **ccard** was created (creation of instance)
 - **ccard** was initialized from credit card infos (modification of attributes)
 - **ccard** was set as **seller's credit card** (creation of association)
 - **ccard** was set as **credit card** for **auc** (creation of association)

instance · **attribute** · **association**

What the contract gives us

The static model

Every noun is a candidate concept: Customer, Cart, Cart Line, Order, Order Line, Product, Stock. Every association created or destroyed is a candidate link between them.

Structure modeling of the domain

The tests

Preconditions are the setup. The command is the action. Postconditions are what to check.

Next lecture: Behavior-Driven Development

Summary

- A **bounded context** is the boundary inside which one model and one ubiquitous language stay consistent.
- Subdomains are discovered, bounded contexts are designed. One context can group several subdomains.
- A **context diagram** shows the contexts, the systems around them, and what crosses each boundary.
- Inside a context, a use case works on the domain model through **application commands**, which take **data transfer objects** as input.
- A **contract** specifies a command with preconditions and postconditions. Every postcondition is one of five kinds of change.